Chapitre 0 — Avant tout | Partie A — Écrire des lettres | leçon 0.10.1 sur 153 | difficulté 1 / 10

0.10.1. Tout l'alphabet, avec poserS — de base, ailleurs

Le 0.10 trois lignes plus bas : tout l'alphabet sur les lignes 3 et 4.

1. Ce qu'il faut comprendre

C'est le 0.10, avec un seul changement : la **ligne** : 3 au lieu de 0. `poserS` passe tout seul à la ligne suivante, la 4.

Le reste ne bouge pas : A à T sur une ligne, U à Z sur la suivante.

C'est la version de base : une seule chose, à la deuxième place. Le 0.10.2 met les deux ensemble.

> **Les fonctions de cette leçon — et le tuto de chacune**
>
> - `#include <ALPHABET>` → 0.1.4. ALPHABET — les lettres de la police
> - `#include <poserS>` → 0.9.3. La fonction poserS() — poser, et passer à la ligne

📖 Les mots de cette leçon — si tu débutes

- **accolades { }** — Elles entourent un bloc : les lignes qui vont ensemble (le corps d'une fonction, d'une boucle, d'un « if »). Chaque « { » a son « } ».
- **affectation (=)** — « x = 5; » : la boîte x prend la valeur 5. Ce n'est pas une égalité : c'est un rangement.
- **argument** — Une valeur donnée à une fonction quand on l'appelle, entre ses parenthèses : dans texte(5, 8, "SALUT"), 5, 8 et "SALUT" sont les arguments.
- **bloc** — Un groupe de lignes entre accolades { }, qui s'exécutent ensemble.
- **booléen (vrai / faux)** — Le résultat d'une condition : vrai (1) ou faux (0). « true » veut dire vrai, « false » faux.
- **boucle** — Des instructions qui se répètent : « while » (tant que…) ou « for » (pour un compteur qui va de… à…).
- **boucle du jeu** — « while (true) { image(); … } » : ce qui se répète 60 fois par seconde, sans fin, pendant tout le jeu.
- **commentaire (//)** — Tout ce qui suit « // » jusqu'au bout de la ligne : une phrase pour l'humain, que la console ignore.
- **condition** — Une question dont la réponse est vrai ou faux : « score > 10 », « bouton(A) ». On la teste avec « if » ou « while ».
- **for** — Une boucle qui compte : « for (uint8_t i = 0; i < 5; i++) » fait 5 tours, i valant 0, 1, 2, 3 puis 4.
- **image()** — Attend l'image suivante. La console en montre 60 par seconde : c'est ce qui règle la vitesse d'un jeu.
- **#include** — En haut du programme : « #include <texte> » demande la fonction texte() à la console ; « #include "fichier.cpp" » verse un de tes fichiers à cet endroit.
- **incrémenter (++)** — « x++ » ou « x += 1 » : x augmente de 1. « x-- » : x diminue de 1.
- **indice** — Le numéro d'une case dans un tableau, entre crochets : cases[3] est la quatrième case (on compte à partir de 0).
- **instruction** — Un ordre donné à la console, terminé par un point-virgule : « texte(1, 1, "OK"); ».
- **main** — « int main() { … } » : là où le programme commence. La console exécute d'abord la première ligne de main.
- **octet** — Un nombre de 0 à 255 : la plus petite boîte de la mémoire. Une cartouche en a 32 768.
- **opérateurs de comparaison** — == égal, != différent, < plus petit, > plus grand, <= plus petit ou égal, >= plus grand ou égal. Le résultat est vrai ou faux.
- **point-virgule (;)** — Il termine une instruction, comme le point termine une phrase.
- **uint8_t** — Le genre de nombre de cette console : un entier de 0 à 255 (un octet). Au-delà, il repart à 0.
- **variable** — Une boîte avec un nom, dans la mémoire, qui retient un nombre qu'on peut lire et changer.
- **while (tant que)** — Répète un bloc tant qu'une condition est vraie.

🔍 Le programme, ligne par ligne — chaque ligne dite en français

`#include <ALPHABET>  // les lettres de la police : ALPHABET[0] est le A`	Demande à la console « ALPHABET » (ce que c'est : les lettres de la police : ALPHABET[0] est le A). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « les lettres de la police : ALPHABET[0] est le A ».)
`#include <poserS>  // pose une tuile, et passe à la ligne au bord`	Demande à la console « poserS » (ce que c'est : pose une tuile, et passe à la ligne au bord). Sans cette ligne, le compilateur refuserait

	de l'employer. (Le commentaire le dit : « pose une tuile, et passe à la ligne au bord ».)
`int main() {  // Le programme commence ici.`	Le début du programme : la console commence ici, par la première ligne qui suit. (Le commentaire le dit : « Le programme commence ici. ».)
`// Le changement : les lignes 0 et 1 → 3 et 4.`	Un commentaire : une phrase pour celui qui lit (« Le changement : les lignes 0 et 1 → 3 et 4. »). La console l'ignore.
`// for : i part de 0, et gagne 1 à chaque tour tant que i < 26 (la taille de ALPHABET).`	Un commentaire : une phrase pour celui qui lit (« for : i part de 0, et gagne 1 à chaque tour tant que i < 26 (la taille de ALPHABET). »). La console l'ignore.
`for (uint8_t i = 0; i < sizeof(ALPHABET); i++) {`	Une boucle qui compte : i part de 0 ; on continue tant que i est plus petit que sizeof(ALPHABET) ; et après chaque tour, i augmente de 1.
`poserS(i, 3, ALPHABET[i]);  // poserS passe tout seul à la ligne 4`	Appel de la fonction poserS de la console, avec les arguments i, 3, ALPHABET[i]. Ce qu'elle fait : comme poser(), mais passe à la ligne tout seul : la colonne 20 devient (0, ligne + 1), comme textS. (Le commentaire le dit : « poserS passe tout seul à la ligne 4 ».)
`// i = la colonne ; à partir de 20, poserS repart à`	Un commentaire : une phrase pour celui qui lit (« i = la colonne ; à partir de 20, poserS repart à »). La console l'ignore.
`// gauche une ligne plus bas (i = 20 → U en (0, 4)).`	Un commentaire : une phrase pour celui qui lit (« gauche une ligne plus bas (i = 20 → U en (0, 4)). »). La console l'ignore.
`// ALPHABET[i] = la lettre n° i (A pour 0, Z pour 25).`	Un commentaire : une phrase pour celui qui lit (« ALPHABET[i] = la lettre n° i (A pour 0, Z pour 25). »). La console l'ignore.
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`while (true) {  // La boucle de jeu : pour toujours...`	La boucle du jeu : tout ce qui est entre les accolades recommence sans fin, 60 fois par seconde avec image(). (Le commentaire le dit : « La boucle de jeu : pour toujours... ».)
`image();  // ... attendre l'image suivante.`	Attendre l'image suivante : la console en montre 60 par seconde. C'est ce qui règle la vitesse du jeu. (Le commentaire le dit : « ... attendre l'image suivante. ».)
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **948 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
  démarrage        49 o  ← la console, à l'allumage  — la mise en route de la console
  main()           74 o  ← démarrage  — ta fonction
  VBlank            8 o  ← la console, à chaque image  — compte les images, 60 fois par seconde
  AttendreImage    28 o  ← image()  — attend l'image suivante
  AttendreVBlank   16 o  ← démarrage  — attend que l'écran soit libre
  AdresseCase      10 o  ← poser()  — trouve l'adresse d'une case de l'écran
  Diviser          17 o  ← poser()  — divise — le processeur ne sait pas le faire
  Reste            10 o  ← poser()  — le reste d'une division
  CopierTuiles     18 o  ← démarrage  — copie les lettres et les dessins en mémoire vidéo
  EffacerCarte     14 o  ← démarrage  — vide les deux cartes de l'écran
```

```
  Tuiles          704 o  ← CopierTuiles  — la police entière
Lettres gravées : toutes (poserS() reçoit un numéro calculé)
Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, EffacerCases, EcrireTexte, EcrireNombre,
Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau,
LireManette, DonneesTransfert, Notes
Fonctions du langage appelées :
  poser()  × 1    49 o sur place, dans main()  + AdresseCase (10 o), Diviser (17 o), Reste (10 o)
  image()  × 1     3 o sur place, dans main()  + AttendreImage (28 o)
  poserS() × 1     0 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <ALPHABET>   // les lettres de la police : ALPHABET[0] est le A
#include <poserS>     // pose une tuile, et passe à la ligne au bord
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

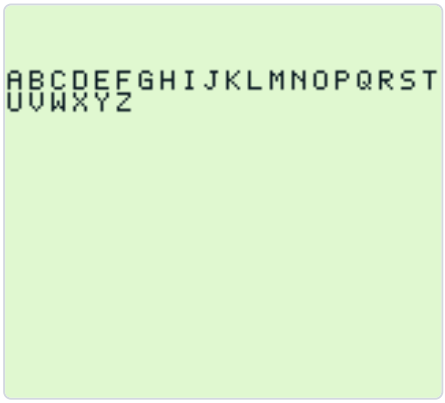
```
int main() {                                // Le programme commence ici.
  // Le changement : les lignes 0 et 1 → 3 et 4.
  // for : i part de 0, et gagne 1 à chaque tour tant que i < 26 (la taille de ALPHABET).
  for (uint8_t i = 0; i < sizeof(ALPHABET); i++) {
    poserS(i, 3, ALPHABET[i]);   // poserS passe tout seul à la ligne 4
                                 // i = la colonne ; à partir de 20, poserS repart à
                                 // gauche une ligne plus bas (i = 20 → U en (0, 4)).
                                 // ALPHABET[i] = la lettre n° i (A pour 0, Z pour 25).
  }

  while (true) {                 // La boucle de jeu : pour toujours...
    image();                     // ... attendre l'image suivante.
  }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.

la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

> Tout l'alphabet sur les lignes 3 et 4.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ ligne 3 : A à T

✓ ligne 4 : U à Z

Ensuite : **0.10.2. Tout l'alphabet, avec poserS — doublé, deux positions** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++